

Chapter 3: Divide and Conquer

*Lecturer: Jasper Lee**Scribe: Jasper Lee*

This chapter explores the first major algorithmic paradigm in this course: Divide and Conquer.

There are primarily two different styles of divide and conquer algorithms:

1. Divide
2. Recurse into 1 piece (e.g. binary search) vs work on every piece then combine (e.g. mergesort)

1 Mergesort and Inversion Counting

We will now look at examples of the second style, starting with mergesort. We recall the following recursive implementation of the algorithm.

Algorithm 1 Mergesort

function SORT(Array A of length n)

1. if $n \leq 1$, return A
 2. return MERGE(SORT($A[1 \dots \lfloor n/2 \rfloor]$), SORT($A[\lfloor n/2 \rfloor + 1 \dots n]$))
-

What mathematical claims and guarantees do we want from SORT and MERGE?

- SORT: on input a numerical array of size n , it should output a sorted permutation of the array in $O(n \log n)$ time.
- MERGE: on input two sorted arrays of sizes m and n , it should output a sorted version of their union/concatenation in $O(m + n)$ time.

We will not explicitly implement and analyze MERGE here, but instead assume the correctness and runtime of MERGE and use it to analyze SORT.

To analyze SORT, we will, of course, use induction. Here, the variable that is changing throughout the computation is the length “ n ” of the input array A , throughout the recursion.

The base case is straightforward: SORT clearly achieve the guarantees if A has length 0 or 1. For the induction step, observe that the output must be a permutation of A : the recursive calls of SORT output permutations of their respective halves by the induction hypothesis, then MERGE preserves the existence of all elements by its correctness guarantee. Lastly, the output permutation must be sorted. This again follows from the induction hypothesis, that the recursive calls to SORT output sorted versions of the array halves, and the correctness of MERGE implies that combining these two sorted halves yields a fully sorted array.

1.1 Counting Inversions

Let us now go beyond mergesort and see a more advanced example: counting inverted pairs.

Definition 3.1. Given an array A , a pair of indices (i, j) is an *inversion* if $i < j$ and $A[i] > A[j]$. Note that i and j are not necessarily adjacent.

The trivial algorithm comparing all pairs runs in $O(n^2)$ time.

How do we apply the divide and conquer paradigm to design a faster algorithm? Here is the key observation.

Any pair $i < j$ *must* either be:

- Both in the first half of the array,
- Both in the second half of the array, or
- i being in the first half and j being in the second half.

This might sound silly and trivial, but this case analysis actually matches the structure of mergesort very well. The first and second cases can be handled by recursive calls like in mergesort, while the last case requires a variant of “merge”. The neat idea here, then, is to count the number of inverted pairs while executing mergesort itself.

Here is an adaptation of “merge” which, on input sorted arrays A and B , counts the number of inversions in $\text{CONCAT}(A, B)$.

Algorithm 2 Count Inversions During Merge

function COUNTINV(Array A of size m , Array B of size n) // Assuming A and B are sorted

1. $a_ind \leftarrow 1$; $b_ind \leftarrow 1$; $count \leftarrow 0$;
 2. while $a_ind \leq m$ and $b_ind \leq n$:
 - (a) if $A[a_ind] \leq B[b_ind]$
 - i. $a_ind \leftarrow a_ind + 1$
 - (b) else
 - i. $count \leftarrow count + (m - a_ind + 1)$
 - ii. $b_ind \leftarrow b_ind + 1$
 3. return $count$
-

We have a while loop in this function, so for proving correctness, we do induction over the loop variable (number of loop iterations). The induction variable and structure is implicit in the following induction hypothesis.

Induction Hypothesis: At the end of every loop iteration, the variable $count$ holds the number of inversions in the (not computationally constructed) array $\text{CONCAT}(A, B[1..(b_ind-1)])$. Moreover, a_ind and b_ind have been non-decreasing up till the end of this loop iteration.

Induction Step: Assume the induction hypothesis holds at the end of the previous loop iteration. We analyze both cases of the if-then-else.

- **Case 1:** a_ind increments. The claim for this induction step is just the previous induction hypothesis, so we are done.
- **Case 2:** b_ind increments, so we have something to actually prove. Let the value of b_ind before the loop iteration be old_b_ind , and the value after be $new_b_ind = old_b_ind + 1$. We know that $A[a_ind] > B[old_b_ind] = B[new_b_ind - 1]$, as this is the “else” case of the if-then-else. Since A is sorted, it follows that for all $k \in [a_ind, m]$, $A[k] \geq A[a_ind] > B[old_b_ind]$. If $a_ind > 1$, then it is also true that $A[a_ind - 1] \leq B[old_b_ind]$, because at the time $a_ind - 1$ was incremented to a_ind , we have $A[a_ind - 1] \leq B[prev_b_ind] \leq B[old_b_ind]$ where $prev_b_ind$ was the b index at the time of a_ind increment. Therefore, if $a_ind > 1$, the number of new inversion pairs introduced from incrementing b_ind is exactly $m - a_ind + 1$. One can also easily check that the same count is correct for the edge case of $a_ind = 1$ as well.

Wrap up: If the loop terminated with $b_ind > n$, then the induction hypothesis implies the claim about COUNTINV. But what if the loop terminates when $b_ind \leq n$? This means the last increment must have been a_ind . So $A[m] \leq B[b_ind]$, and consequently everything in A is smaller than or equal to everything in $B[b_ind..n]$. So the number of inversions in $\text{CONCAT}(A, B)$ is equal to the number of inversions in $\text{CONCAT}(A, B[1..(b_ind - 1)])$, proving the claim.

2 Longest Monochromatic Path in a Binary Tree

Consider a binary tree of size n , where each vertex is colored either red or black. We want to find the longest length among all-red paths (where length is defined as the number of nodes).

To apply the Divide and Conquer paradigm, we observe a key structural property.

Observation 3.2. Any longest red path within a subtree rooted at some node v must either:

1. Be fully contained within the left child’s subtree,
2. Be fully contained within the right child’s subtree, or
3. Pass through the root v (which requires v to be red, and concatenates longest red paths starting downwards from the left and right children).

2.1 A First Approach

This structural observation naturally motivates a divide-and-conquer algorithm: recursively compute the longest red path lengths in the left and right subtrees for the first two cases, then handle the third case with DFS that traverses downwards from any given node to compute the longest red path length starting that node. Finally, we take the maximum across the three cases.

Algorithm 3 Divide and Conquer

function DFS(Tree *node*)

1. if *node* is NULL or *node* is black, return 0
2. return $1 + \max(\text{DFS}(\text{node.left}), \text{DFS}(\text{node.right}))$

function LONGESTPATH(Tree *root*)

1. if *root* is NULL, return 0
 2. $\text{len_left} \leftarrow \text{LONGESTPATH}(\text{root.left})$
 3. $\text{len_right} \leftarrow \text{LONGESTPATH}(\text{root.right})$
 4. if *root* is red
 - (a) $\text{len_through_root} \leftarrow \text{DFS}(\text{root.left}) + 1 + \text{DFS}(\text{root.right})$
 5. else
 - (a) $\text{len_through_root} \leftarrow 0$
 6. return $\max(\text{len_left}, \text{len_right}, \text{len_through_root})$
-

If we naively call this linear time DFS helper function on the current root at every recursive step, we obtain the following recurrence for the overall runtime $T(n)$:

$$T(n) \leq T(n_L) + T(n_R) + O(n)$$

where n_L and n_R are the sizes of the left and right subtrees, respectively, satisfying $n_L + n_R = n - 1$.

For *balanced* binary trees, where $n_L \approx n_R \approx n/2$, this yields the recurrence:

$$T(n) \leq 2T(\lceil n/2 \rceil) + O(n)$$

This is the same recurrence as for mergesort, which we know takes $O(n \log n)$ time.

For unbalanced trees, the runtime actually gets worse! For example, consider a “binary tree” that is just a single long linked list, meaning $n_L = n - 1$ and $n_R = 0$. The runtime recurrence becomes:

$$T(n) \leq T(n - 1) + O(n)$$

which solves to as much as $O(n^2)$ time, at least in badly behaved trees.

Remark Divide and Conquer algorithms (and algorithms generally) can take runtime that depends not only on the size of the input, but also on *how* the input is structured. This algorithm illustrates the dependence that can happen.

2.2 An Optimal Linear Time Algorithm

The bulk of the computation in the previous algorithm came from the 2 DFS calls within each node. To improve the algorithm down to optimal linear time, the observation is that

the DFS calls have the same structure as the overall recursion, so the DFS calls are actually *redundant*.

For the new algorithm, instead of having the recursive call return one quantity (length of longest red path through a given vertex), we will ask it to compute *two* pieces of information:

1. Length of longest red path in the subtree rooted at the input vertex.
2. Length of longest red path in the subtree that has an endpoint at the input vertex.

By passing both values up the recursion, we can compute the longest red path length through the root (third case of case analysis) in $O(1)$ time rather than $O(n)$ time.

Algorithm 4 Linear Time Longest Monochromatic Path

function NEWLONGESTPATH(Tree *root*)

1. if *root* is NULL, return (0, 0)
 2. (*len_left*, *len_left_root*) $\leftarrow$ NEWLONGESTPATH(*root.left*)
 3. (*len_right*, *len_right_root*) $\leftarrow$ NEWLONGESTPATH(*root.right*)
 4. if *root* is red
 - (a) *len_through_root* $\leftarrow$ *len_left_root* + 1 + *len_right_root*
 - (b) *len_root* $\leftarrow$ max(*len_left_root*, *len_right_root*) + 1
 5. else
 - (a) *len_through_root* $\leftarrow$ 0
 - (b) *len_root* $\leftarrow$ 0
 6. return (max(*len_left*, *len_right*, *len_through_root*), *len_root*)
-

With this modification, the recursive work done at each node is strictly bounded by a constant, implying that the algorithm runs in $O(n)$ time.

While the above informal argument is sufficient for your homeworks and exams, more formally, we can always form the new runtime recurrence relation:

$$T(n) \leq T(n_L) + T(n_R) + O(1)$$

where again $n_L + n_R = n - 1$. By induction, it is straightforward to show that $T(n) \leq O(n)$.

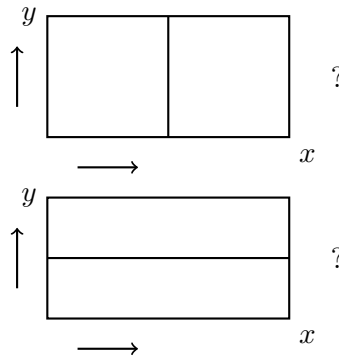
In fact, the above algorithm can be generalized for trees that are not only binary ones, which still runs in $O(n)$ time. There is a way to phrase the generalized algorithm such that, in each recursive call, the amount of work (in addition to making the recursive calls) is upper bounded by $O(\# \text{ children})$.

This also segues nicely to introducing a simpler way to argue about the runtime of recursive algorithms without resorting to the complete formality of induction. Notice that there is one recursive call per vertex in the tree, and the amount of work per vertex is $O(\# \text{ children})$. Thus, the total runtime is $O(\sum_v \# \text{ children for } v)$, but this sum is just the number of vertices in the tree! So we can conclude that the generalized algorithm has $O(n)$ runtime.

3 Closest Pair of Points in 2D

Given a list of n points (x_i, y_i) , we want to find the pair that is closest to each other, or at least, the distance of that closest pair. The trivial algorithm of trying all pairs is $O(n^2)$. We want to improve this to $O(n \log n)$ time.

For divide and conquer, we have to decide on how to divide. Since the problem lives on the x, y -plane, naturally we can choose to divide along either axis.



To yield an $O(n \log n)$ time algorithm, we really want to divide the n points in two halves of $n/2$ and $n/2$. We can achieve that by splitting points using the median x -coordinate (or y -coordinate, either works).

As we are seeing more and more examples of Divide and Conquer algorithms, hopefully a pattern is emerging. At the core of the algorithm, there is usually a key structure observation that enables the “conquer” step, and the observation is commonly a case analysis related to how we “divide”.

For the closest pair of points problem, we make the following observation. Assuming we split by the x -coordinate median, the closest pair must either be:

1. Both in the left half (solved by recursion).
2. Both in the right half (solved by recursion).
3. One point is in the left half, and one is in the right half (requires additional work for the “conquer” step).

At this point, we have an outline of what the algorithm should look like:

Algorithm 5 Closest Pair Algorithm Outline

function CLOSEST(List A)

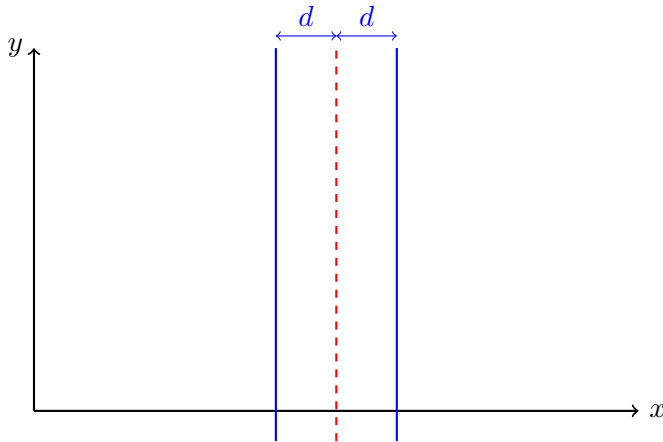
1. Find median in x , split into left and right halves.
 2. $d_L \leftarrow \text{CLOSEST}(\text{left half})$
 3. $d_R \leftarrow \text{CLOSEST}(\text{right half})$
 4. Compute d_{CROSS} for the cross-boundary case.
 5. return $\min(d_L, d_R, d_{\text{CROSS}})$
-

So then the big question is, how do we compute d_{CROSS} ?

If we compute d_{CROSS} naïvely, it takes $O(n^2)$ time *per recursive call* (although n does halve every call), making the total runtime at least $\Omega(n^2) \gg O(n \log n)$.

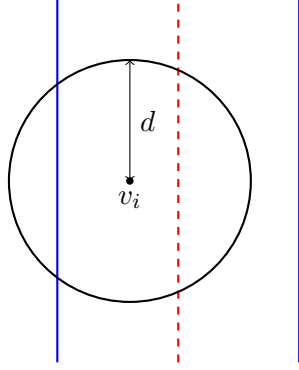
What we will see is a clever geometric algorithm that computes d_{CROSS} in $O(n)$ time, which yields an overall recursive $O(n \log n)$ time algorithm. The algorithm is highly non-trivial, but it is good to see an example of a harder algorithm.

Let $d = \min(d_L, d_R)$. Since we are only interested in pairs that are strictly closer than d , we only need to consider points within a vertical strip of x -median $\pm d$.

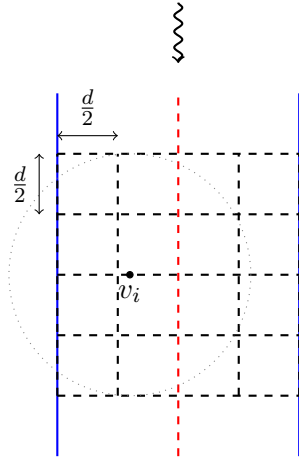


This basic observation is not enough yet, since every point might lie along this strip, so a naïve enumeration will still take $\Omega(n^2)$ time.

So here is the next observation. For any specific point v_i inside the strip, any point within distance d must lie in a radius- d ball around v_i . Meaning, for point v_i , we only care about the intersection of this ball with the strip.



We only need to consider the radius- d ball around v_i (specifically, its intersection with the strip).



This 4×4 grid *covers* the entire radius- d circle around v_i intersecting with the strip, so it is sufficient to check this area.

At most 1 point can lie in each square, since two points in the square have distance at most $d/\sqrt{2} < d$.

So we only need to consider the **15 closest points** in the y -coordinate within the strip.

In fact, observe that every pair (v_i, v_j) is being considered a duplicate second time as (v_j, v_i) . We can therefore optimize the 15-point check to a 7-point check, by sorting the points in the strip by their y -coordinates and pairing each point v_i with the next 7 points in the sorted list. This is almost an $O(n)$ merge step, except, the y -sorting requires $O(n \log n)$ time. This unfortunately yields an $O(n \log^2 n)$ time algorithm generally.

Algorithm 6 Intermediate $O(n \log^2 n)$ Closest Pair

function CLOSEST(List A)

1. Find median x -coordinate, split into left, right halves (Takes $O(n)$ time, but naïve sorting takes $O(n \log n)$ time)
 2. $d_L \leftarrow \text{CLOSEST}(\text{left half})$
 3. $d_R \leftarrow \text{CLOSEST}(\text{right half})$
 4. Sort points in A by y -coordinate (Takes $O(n \log n)$ time)
 5. New list $A' \leftarrow \text{filter } A \text{ to strip of } \min(d_L, d_R)$
 6. For each v_i in A' , try pairing with next 7 points to compute d_{CROSS}
 7. return $\min(d_L, d_R, d_{\text{CROSS}})$
-

If only we can sort in $O(n)$ time! (But that is impossible, at least for sorting algorithms)

based on comparing pairs of elements.)

The final observations are that: for the x -coordinate, we can just pre-sort the entire original input array; for the y -coordinate sorting, the recursive structure of CLOSEST is exactly the same as mergesort (noting that the “divide” step in mergesort can in fact arbitrarily bi-partition a list, instead of only considering the two contiguous halves). So the final algorithm would compute CLOSEST and mergesort in the y -coordinate all at the same time. It is an exercise for you to figure out the details.

4 Strassen’s Algorithm for Matrix Multiplication

From this section onwards, we move on to numerical and linear algebraic algorithms that are based on divide and conquer.

First, we consider the matrix multiplication problem. Given two $n \times n$ matrices A and B , we want to compute their product $C = AB$. The naïve computation, which is how we do matrix multiplication by hand, takes $O(n^3)$ time.

Matrix multiplication inherently has a divide and conquer structure. The operation can be done in a blockwise manner as follows: interpreting A and B into a 2×2 matrix of matrix *blocks*, then the following holds.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

This block formulation requires 8 matrix multiplications for size $\frac{n}{2} \times \frac{n}{2}$, so the runtime recurrence is $T(n) = 8T(\lceil n/2 \rceil) + c_1 n^2$. Solving this recurrence unfortunately yields $T(n) = \Theta(n^3)$, which is not helping at all.

So why is the block structure useful? Strassen made an observation that is, frankly, magical. It turns out there is a way to compute 2×2 matrix multiplication using only 7 multiplications as opposed to 8!

First compute:

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22})B_{11} \\ M_3 &= A_{11}(B_{12} - B_{22}) \\ M_4 &= A_{22}(B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{12})B_{22} \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}) \end{aligned}$$

Then the quadrants of C are reconstructed as:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}$$

Why does it matter that we only need 7 instead of 8 (matrix) multiplication recursive calls? The new recurrence is now $T(n) \leq 7T(\lceil n/2 \rceil) + cn^2$. It is not difficult to show by

induction that $T(n) \leq c'n^{\log_2 7} - c''n^2$ (for some sufficiently large c' and c''). Noting that $\log_2 7 < \log_2 8 = 3$, Strassen's algorithm is strictly faster than $O(n^3)$ time.

Here is a final fun exercise. Given two complex numbers $a + bi$ and $c + di$, their product is $(ac - bd) + (ad + bc)i$. Naïvely, computing this product requires 4 real-number multiplications (ac , bd , ad , and bc). Can you compute this product using only 3 real-number multiplications?

5 Integer Multiplication (Karatsuba's Algorithm)

Setting: Given two n -bit integers x and y , we want to compute their product $z = xy$. The standard algorithm for this is the one we learn in elementary school (applied to binary), which takes $O(n^2)$ time.

$$\begin{array}{r} 1 1 0 1 \\ \times 0 1 0 1 \\ \hline 1 1 0 1 \\ 0 0 0 0 \\ 1 1 0 1 \\ + 0 0 0 0 \\ \hline 1 0 0 0 0 1 \end{array}$$

Can we do better, like Strassen's Algorithm?

The first step is to look at integer multiplication from a Divide and Conquer perspective. We can split the bits of x and y into halves:

$$x = x_{\text{low}} + 2^{\lfloor n/2 \rfloor} x_{\text{high}}$$

$$y = y_{\text{low}} + 2^{\lfloor n/2 \rfloor} y_{\text{high}}$$

where $x_{\text{low}} = x \bmod 2^{\lfloor n/2 \rfloor}$ (the bottom bits) and $x_{\text{high}} = x \operatorname{div} 2^{\lfloor n/2 \rfloor}$ (computed via a bit shift).

The product can be expanded as:

$$xy = x_{\text{low}}y_{\text{low}} + 2^{\lfloor n/2 \rfloor}(x_{\text{high}}y_{\text{low}} + y_{\text{high}}x_{\text{low}}) + 2^{2\lfloor n/2 \rfloor}x_{\text{high}}y_{\text{high}}$$

The simple method computes these 4 products ($x_{\text{low}}y_{\text{low}}$, $x_{\text{high}}y_{\text{low}}$, $y_{\text{high}}x_{\text{low}}$, $x_{\text{high}}y_{\text{high}}$) recursively, and then does bit shifts to compute the final product with $2^{\lfloor n/2 \rfloor}$ and $2^{2\lfloor n/2 \rfloor}$. The recurrence for this simple method is:

$$T(n) \leq 4T(n/2) + O(n)$$

which yields $T(n) \leq O(n^2)$.

How do we improve this to 3 multiplications? This was the exercise from the last section, expressed in terms of complex numbers. We can use a simpler version of Strassen's trick:

- Compute $a = x_{\text{low}}y_{\text{low}}$
- Compute $b = x_{\text{high}}y_{\text{high}}$
- Compute $c = (x_{\text{low}} + x_{\text{high}})(y_{\text{low}} + y_{\text{high}})$

Then, the middle term can be recovered by

$$x_{\text{high}}y_{\text{low}} + y_{\text{high}}x_{\text{low}} = c - a - b$$

This reduces the algorithm to 3 recursive multiplications. The new recurrence is:

$$T(n) \leq 3T(n/2) + \alpha n$$

By induction, we can prove that $T(n) \leq n^{\log_2 3} - \beta n$ for some constant $\beta \gg \alpha$, which gives an overall runtime of $O(n^{\log_2 3})$.

6 Polynomial Multiplication

Setting: Given two degree- n polynomials

$$p(x) = p_0 + p_1x + \cdots + p_nx^n$$

$$q(x) = q_0 + q_1x + \cdots + q_nx^n$$

Compute the product $(p \cdot q)(x)$.

We can use the exact same algorithm! Let $p_{\text{low}}(x)$ be the terms of p up to degree $\lfloor n/2 \rfloor$, and define the higher-degree terms as:

$$p_{\text{high}}(x) = \frac{p(x) - p_{\text{low}}(x)}{x^{\lfloor n/2 \rfloor + 1}}$$

We can express the product $p \cdot q$ identically to integer multiplication:

$$p \cdot q = p_{\text{low}}q_{\text{low}} + x^{\lfloor n/2 \rfloor + 1}(p_{\text{low}}q_{\text{high}} + q_{\text{low}}p_{\text{high}}) + x^{2\lfloor n/2 \rfloor + 2}p_{\text{high}}q_{\text{high}}$$

Using the same 3-multiplication trick, this has exactly the same structure as Karatsuba's algorithm before, so we have an $O(n^{\log_2 3})$ algorithm.

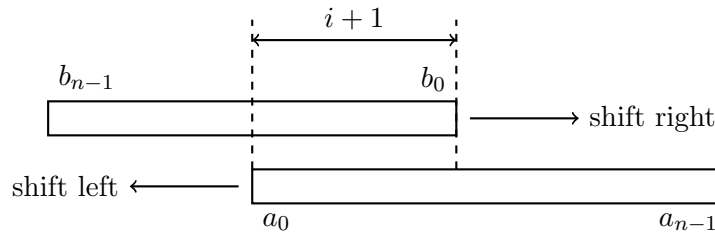
Can we do even better? **YES!** We can improve this basically to $O(n \log n)$.

7 Convolution and Fast Fourier Transform

Setting: Given two length- n sequences a and b , compute the convolution $a * b$ (of length $2n - 1$).

$$(a * b)_i = \sum_{j=0}^i a_j b_{i-j}$$

Visually, this can be understood as reversing array b and “swiping” it across array a . $(a * b)_i$ corresponds to an overlap of width $i + 1$ and computing the dot product across the overlapping elements.



The naïve way to compute this takes $O(n^2)$ time: $O(n)$ terms each taking $O(n)$ work (for the dot product). We can improve this to $O(n \log n)$ time using the *Fast Fourier Transform* (FFT) algorithm.

7.1 Discrete Fourier Transform

Before discussing the Fast Fourier Transform which is an algorithm, we need to introduce the Discrete Fourier Transform (DFT) which is the mathematical operation that the FFT computes.

Given a length- n sequence (vector) v , the Discrete Fourier Transform of v is the matrix-vector product $\mathcal{F}_n v$, where $\mathcal{F}_n$ is defined as the following $n \times n$ matrix:

$$\mathcal{F}_n = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)^2} \end{pmatrix}$$

where ω is a primitive n -th root of unity (i.e. $\omega^n = 1$ and $\omega \neq 1$), for example $\omega = e^{-2\pi i/n}$. Computing the DFT naïvely via standard matrix-vector multiplication $\mathcal{F}_n v$ takes $O(n^2)$ time.

Taking Fourier Transforms are standard procedures, with different standard libraries containing implementations.

How are Discrete Fourier Transforms related to convolutions? To discuss this, we need to first introduce the inverse transform $\mathcal{F}_n^{-1}$, which is mathematically very similar. It is essentially the same as the forward Fourier Transform, but instead using the inverse of ω and scaling by $\frac{1}{n}$:

$$\mathcal{F}_n^{-1} = \frac{1}{n} \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \dots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \dots & \omega^{-2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \dots & \omega^{-(n-1)^2} \end{pmatrix}$$

The Discrete Fourier Transform has a powerful and convenient property, relating to convolutions: convolution in the normal domain corresponds to pointwise multiplication in the Fourier domain¹:

$$a * b = \mathcal{F}_n^{-1} ((\mathcal{F}_n a) \cdot (\mathcal{F}_n b))$$

Since pointwise multiplication of two sequences takes linear time, if we can compute forward and inverse DFTs efficiently, we can compute convolutions efficiently. The Fast Fourier Transform (FFT) algorithm, described in the next section, lets us compute DFTs and hence convolutions in $O(n \log n)$ time. But an important note is that the Fourier Transform is numerically quite unstable; in fact, MATLAB implements convolutions naïvely (last time I checked) instead of using the FFT.

So why is this useful for polynomial multiplication? It turns out polynomial multiplication is just a convolution:

$$p \cdot q = p * q$$

¹Strictly speaking, computing this with an $n \times n$ DFT matrix yields the *circular* convolution different from the *linear* convolution above, so the equivalence is technically false. But this can be made true via sufficient zero-padding, so the claim as stated is “morally true” and is representative of the general duality between convolution and pointwise multiplication in the normal and Fourier domains.

$$(p \cdot q)(x) = \sum_i x^i \sum_j p_j q_{i-j}$$

That means we can also do polynomial multiplication in $O(n \log n)$ time, using FFT below.

7.2 Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is a standard divide and conquer algorithm that leverages the structure of the DFT matrix and computes $\mathcal{F}_n v$ in $O(n \log n)$ time, as opposed to the slower naïve $O(n^2)$ time.

By definition, the k^{th} element of the DFT is:

$$(\mathcal{F}_n v)_k = \sum_{j=0}^{n-1} v_j e^{-i \frac{2\pi k j}{n}}$$

We can split the sum into even and odd indices, and then manipulate the two sums slightly:

$$\begin{aligned} (\mathcal{F}_n v)_k &= \sum_{j=0}^{\frac{n}{2}-1} v_{2j} e^{-i \frac{2\pi k (2j)}{n}} + \sum_{j=0}^{\frac{n}{2}-1} v_{2j+1} e^{-i \frac{2\pi k (2j+1)}{n}} \\ &= \sum_{j=0}^{\frac{n}{2}-1} v_{2j} e^{-i \frac{2\pi k j}{n/2}} + e^{-i \frac{2\pi k}{n}} \sum_{j=0}^{\frac{n}{2}-1} v_{2j+1} e^{-i \frac{2\pi k j}{n/2}} \end{aligned}$$

Now the key observation is that the two sums are exactly the k^{th} elements of the DFTs of the even and odd halves of v , each of length $n/2$, which can be handled by recursive FFT calls. In other words, computing the DFT of size n requires exactly two DFTs of size $n/2$, and a final pointwise multiplication by $e^{-i \frac{2\pi k}{n}}$ (taking $O(n)$ time).

The runtime recurrence relation is

$$T(n) \leq 2T(n/2) + O(n)$$

which implies $T(n) \leq O(n \log n)$.